Learn Spring Boot through QuickSlot

A practical roadmap for someone who already knows Java

20 October 2026 | Maintained alongside the QuickSlot rebuild

You already know Java. The next step is understanding how a Spring application receives a web request, checks permissions, runs business rules and commits data safely. Learn those boundaries through this project rather than collecting annotations without context.

QuickSlot uses a React interface, a Java Spring Boot backend and PostgreSQL. The backend is a modular application, not a set of microservices. Java libraries handle PDF and Word timetable input; no Python service is required.

This guide is a learning plan, not a claim that every production capability has been delivered. Check the repository README and test results for implementation status. Examples labelled illustrative explain concepts and may differ from final class names.

How to use the guide

- Read one chapter, inspect the related feature in the repository, then complete its exercise.
- Explain the request flow aloud before adding another framework dependency.
- Keep one learning branch for experiments; do not weaken production checks just to make a demo succeed.
- Update this guide whenever the implementation changes its configuration, security or persistence approach.

Learning order

1. Application startup, Maven and dependency injection.
2. HTTP controllers, request records, validation and errors.
3. PostgreSQL, JDBC and schema migrations.
4. Transactions and competing room allocations.
5. Login, CSRF and scoped permission checks.
6. Timetable evidence, review and publication.
7. Tests that exercise failures and races.
8. Configuration, deployment and operational ownership.

Finish by demonstrating one complete request: a club head submits, the coordinator forwards, and the HOD approves after the system rechecks the room.

Understand startup and dependency injection

What you need to learn

Spring Framework manages application objects called beans. Spring Boot adds application setup, conditional configuration and an embedded web server. Maven resolves declared dependencies and runs compilation, tests and packaging. A starter brings a compatible group of libraries; it is not a separate server you must install.

Learn what @SpringBootApplication does at a practical level: configuration, component scanning and auto-configuration. Keep the application class above feature packages so scanning finds their components. A class outside that package tree may need explicit registration.

Constructor injection means a service receives its collaborators through its constructor. Spring constructs the graph. Your business code should not fetch a bean from a global container or construct its own database connection.

```
// Illustrative dependency injection example
@Service
public class ApprovalService {
    private final JdbcTemplate jdbc;

    public ApprovalService(JdbcTemplate jdbc) {
        this.jdbc = jdbc;
    }
}
```

Connect it to QuickSlot

Find the application entry point and follow one controller's constructor to its service. Then find where the DataSource is configured. The JDBC helper depends on that DataSource; the service depends on the helper. This is ordinary Java object composition with the framework doing the wiring.

Exercise and completion check

- Build and run the application using its documented Maven command.
- Read the startup log and identify the port, active profile and database initialization result.
- Add a small injected clock to an exercise-only service so you can test time-dependent rules without changing the system clock.
- Explain why a static mutable map of all booking state would break when two application instances run.

Avoid field injection and unnecessary interface/implementation pairs. Introduce an interface when there is a useful boundary or more than one implementation, not because every class must have one. Read reference 1 for dependency injection and reference 2 for Boot setup.

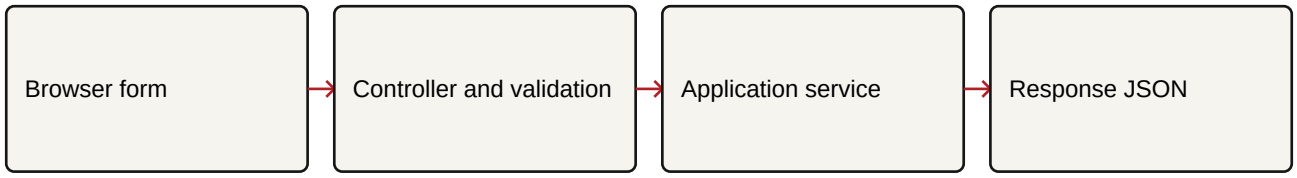
Follow an HTTP request

What you need to learn

A controller translates HTTP into a use case. @GetMapping and @PostMapping associate paths and methods with handlers. @RequestParam reads query values, @PathVariable reads a path segment, and @RequestBody reads JSON. Records work well for small request/response types.

Bean Validation checks input shape with annotations such as @NotBlank and @Positive; @Valid activates validation at the controller boundary. Business rules still belong in the service: a positive attendance count can exceed a particular room's capacity.

Jackson converts Java objects to JSON. QuickSlot's frontend expects snake_case field names. Keep this contract explicit and test it; a backend rename must not silently break a form.



A QuickSlot example

A request contains a room ID, date, start/end times, purpose and attendance. The server derives the requester from the authenticated session. The client must not choose its own user ID, role or approval authority.

Use LocalDate and LocalTime for user-entered campus values, then combine them with Asia/Kolkata when creating concrete events. Use OffsetDateTime or Instant for actual stored instants. Preserve room code 017 as text.

Exercise and completion check

- Trace a request from the React API client to a Java handler and back to the rendered status.
- Submit malformed dates, a blank purpose, reversed times and oversized attendance. Check both status and useful error message.
- Distinguish 401 (no valid login), 403 (no permission), 404 (unavailable object), and 409 (conflicting current state).
- Confirm that public search responses never include private application purpose or requester email.

Learn centralized exception handling with @RestControllerAdvice. Log technical details on the server without leaking SQL or stack traces to a user. Reference 3 introduces REST controllers; reference 4 covers validation.

Learn persistence before ORM shortcuts

What you need to learn

A DataSource manages database connections. JdbcTemplate handles repetitive JDBC resource management while letting you write SQL explicitly. Query parameters must be bound, never concatenated from input. Transactions let several writes succeed or fail together.

QuickSlot starts with Spring JDBC and PostgreSQL. JPA/Hibernate are useful technologies but are not prerequisites for understanding this implementation. Learn them later if a real feature benefits from object-relational mapping.

Flyway records schema migration history. Versioned SQL files create or change tables in order. Once a migration has been applied to a shared environment, add a new migration instead of editing its history. Never use automatic destructive schema generation as a deployment strategy.

What the database must represent

- Users and durable sessions; room inventory and allocation assignments.
- Requests, forwarding/decision history and approved room occupancy.
- Timetable sources, review revisions and effective publication windows.
- Audit records and durable in-app notifications.

Use constraints for rules the database can enforce: foreign keys, valid time ranges, unique identities and non-overlapping active room reservations. The application still checks scope, capacity and schedule coverage.

Exercise and completion check

- Apply migrations to an empty disposable database and inspect the resulting tables.
- Write one parameterized query to list a user's own requests.
- Explain why a query that finds no current booking does not prove complete timetable coverage.
- Add a migration on an exercise branch and confirm both a new database and an existing database reach the same schema.

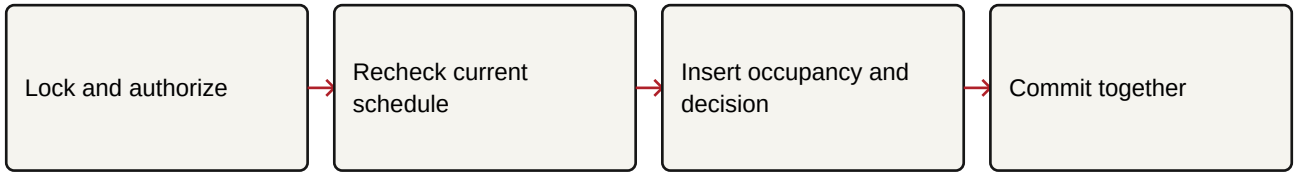
Learn query plans and indexes after understanding the queries. Measure slow paths before adding a cache. Reference 5 explains JdbcTemplate; reference 6 explains migration ordering.

Make booking decisions atomic

What you need to learn

@Transactional places a transaction boundary around a Spring-managed method. A database transaction is not automatically a lock on every row another request might create. Two transactions can both observe an apparently empty room.

For scheduling, combine short write transactions, a consistent lock protocol, fresh checks and a PostgreSQL exclusion constraint. Half-open intervals [start,end) allow one class to end when the next starts; partial overlaps must still conflict.



QuickSlot invariants

Pending and forwarded requests do not reserve space. Only a successful approval creates occupancy. Approval must recheck the latest timetable, coverage, capacity and room access. A database conflict must roll back occupancy, status, audit and notification changes together.

Every timetable publisher and booking writer must follow the same coordination protocol. A database lock coordinates application instances; a Java synchronized block only coordinates threads in one process. A broad lock can be a safe initial simplification but limits throughput. Narrowing it requires measured tests and consistent ordering.

Spring pitfalls to understand

- Calls made inside the same object may bypass a proxy-based @Transactional boundary. Put the transaction at an externally invoked service method.
- Know rollback rules. Do not catch an exception and accidentally commit a partially completed operation.
- Do not hold scheduling locks while parsing a PDF, sending email or calling another website.
- Retrying a submission needs persistent idempotency, not merely a disabled button.

Exercise and completion check

Run two independent database connections attempting the same room/time. Verify that exactly one conflicting occupancy commits. Test adjacent slots too. Inspect stored rows, not just HTTP responses. Reference 7 explains Spring transaction behavior; reference 8 covers PostgreSQL range exclusion.

Understand login and scoped authority

What you need to learn

Authentication answers who signed in. Authorization answers whether that user may act on this object now. A role label alone is insufficient: a coordinator must be responsible for the requested room, and a HOD must have the relevant department scope.

A server-side session uses an opaque browser cookie and durable server state. Store a hash of the random session token. The browser cookie should be HttpOnly and Secure over HTTPS, with an appropriate SameSite policy. Do not put passwords or bearer tokens in browser local storage.

Cookie authentication needs CSRF protection for state-changing requests. Learn how the browser sends the CSRF header and how the server validates it. CORS and CSRF solve different problems. A hidden button is not an authorization check.

QuickSlot permission exercise

- Anonymous visitors and students may inspect permitted classroom/LT availability.
- Faculty and verified club heads may submit booking requests; students may not.
- The assigned coordinator may approve, reject or forward a request.
- A forwarded request requires a HOD decision. Its coordinator may still inspect it.
- An administrator manages sources and accounts but does not gain approval powers automatically.
- A requester cannot approve their own request, even when otherwise authorized.

Exercise and completion check

Call the API directly as each role. Change a room ID to one outside the user's scope. Send no CSRF header, then an incorrect one. Deactivate an account and verify an existing session stops working. Confirm demo credentials cannot be created accidentally in a production profile.

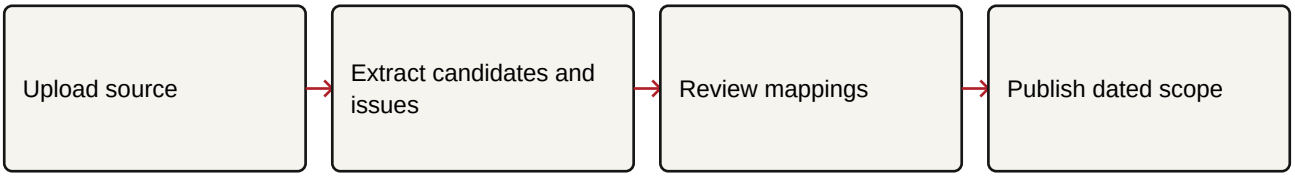
Learn Spring Security's filter chain, password encoders and session concepts even if some domain checks remain explicit service methods. Reference 9 introduces web security; reference 10 explains CSRF.

Treat timetables as reviewed evidence

What you need to learn

Multipart uploads carry binary files. Validate both size and supported structure. Apache PDFBox exposes PDF text positions; Apache POI exposes DOCX tables and document metadata. Text extraction does not resolve the meaning of electives or shared classes.

Keep extracted candidates separate from reviewed events. An empty timetable cell means no listed event for that particular group; it does not establish that a room is free across departments.



Review and replacement

Retain source bytes, digest and cell/page evidence. Preserve room codes as strings. A merged lab session needs its whole interval. Ambiguous room identities, faculty alternatives and electives stay unresolved. Do not quietly assign the first matching room.

A publication has a scope and effective dates. Replacing CSE semester V must preserve semester III and other courses. Keep old dates reproducible. Reject a replacement that conflicts with a confirmed booking until someone explicitly resolves the conflict.

Concurrent administrators need stale-edit protection. A review or publish command carries the revision it was based on. If another edit wins first, reject the stale command and ask the user to refresh.

Exercise and completion check

- Compare one extracted lecture and one merged lab against the original PDF.
- Upload an incomplete Word template and verify it remains a draft with issues.
- Publish two independent scopes, replace one, then check both historical and future dates.
- Explain why a newer upload timestamp does not establish a newer effective timetable.

Learn a durable background job/worker model before allowing large imports at university scale. A bounded foreground parser is a prototype tradeoff, not equivalent to crash-recoverable processing. References 11 and 12 cover the Java document libraries.

Test behavior rather than annotations

What you need to learn

JUnit runs tests. MockMvc exercises HTTP behavior without requiring a separate browser. @SpringBootTest can start a fuller application context. Integration tests must use real PostgreSQL for exclusion constraints and concurrency; an in-memory substitute cannot establish those guarantees.

Keep pure interval calculations fast and separate from database tests. Test the request/response contract and authorization through real handlers. Prefer an actual failure scenario to a mock that merely confirms a method was called.

QuickSlot acceptance checklist

- Student submission is forbidden; faculty and club-head submissions succeed.
- Coordinator forwarding changes decision authority without reserving a room.
- HOD approval rechecks current conflicts, and self-approval is denied.
- Incomplete schedule coverage produces Unknown, never verified Available.
- Two competing approvals cannot both allocate the same interval.
- Duplicate retries do not create duplicate requests or decisions.
- Replacing one timetable preserves other scopes and historical dates.
- Draft source files remain private; public searches reveal no private purpose.

Exercise and completion check

Break one authorization check intentionally on an exercise branch and confirm a test catches it. Then restore the check. Try the same with an interval boundary. This demonstrates that the tests protect behavior rather than merely passing alongside the code.

Use disposable databases or uniquely named test schemas. Never point cleanup code at a university database. Record which tests ran, which were skipped and what they establish. A successful frontend build does not test a booking race. Reference 13 introduces Spring's test support.

Operate and maintain the application

What you need to learn

Configuration belongs outside compiled code. Understand application properties, environment variables and profiles. Keep local demonstration settings distinct from production. Maven produces a deployable JAR; the embedded web server runs inside the Java process.

A reverse proxy serves the React build and forwards /api requests to the backend. HTTPS terminates at the proxy. PostgreSQL stays on a private network. A single well-operated deployment is more useful here than unnecessary microservices.

Before real university use

- Replace demo accounts with an approved provisioning and club-head verification process.
- Confirm timetable ownership, full room coverage, holiday handling and replacement policy.
- Back up both database records and retained sources; rehearse restoration into a separate database.
- Collect request IDs, failed imports, permission failures, conflict rates and database wait times.
- Bound parser resource use, submission rates and database queries; measure response time with a declared dataset.
- Document cancellation, coordinator absence and HOD delegation policies.

What to postpone

Kafka, Kubernetes, reactive programming, native images and a microservice split are optional technologies. Add them only when a measured need justifies their operational cost. Caching can improve search performance but must never authorize an allocation.

Final demonstration

Explain one complete user journey, trace its classes and SQL, show an expected failure, and explain how the system recovers. You are ready to maintain that feature when you can explain its boundaries without relying on annotation names alone.

Maintaining this guide

For each milestone, add what changed, the relevant source package, one exercise and the test that proves the rule. Keep the roadmap synchronized with actual code. Record remaining release gates honestly.

Official reading list

Use the repository's pinned versions when following examples. Some links show the newest documentation; check version selectors before copying code into a Spring Boot 3.5 project.

1. Dependency injection - https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html

2. Spring Boot requirements - https://docs.spring.io/spring-boot/3.5/system-requirements.html

3. REST service guide - https://spring.io/guides/gs/rest-service/

4. Spring validation - https://docs.spring.io/spring-framework/reference/core/validation/beanvalidation.html

5. JDBC core classes - https://docs.spring.io/spring-framework/reference/data-access/jdbc/core.html

6. Flyway versioned migrations - https://documentation.red-gate.com/flyway/flyway-concepts/migrations/versioned-migrations

7. Transaction annotations - https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/annotations.html

8. PostgreSQL range constraints - https://www.postgresql.org/docs/current/rangetypes.html#RANGETYPES-CONSTRAINT

9. Securing a web application - https://spring.io/guides/gs/securing-web/

10. CSRF protection - https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html

11. Apache PDFBox - https://pdfbox.apache.org/

12. Apache POI - https://poi.apache.org/

13. Spring testing - https://docs.spring.io/spring-framework/reference/testing.html

Club membership and verified room details

What you need to learn

A verified club membership is a relationship between a student and a club. It is separate from the account role. A student who joins Co-Dev remains a student and cannot book a classroom. Model that distinction with club_memberships and club_managers instead of rewriting users.role.

Public club profiles show the supplied organizing committee data. Membership requests and reviewer lists need a signed session. A reviewer must be an assigned club head or a same-campus administrator. Every write still needs CSRF protection.

Read the implementation

Follow ClubController into ClubService. Join requests lock the account before inserting the unique club/student relationship, which prevents two repeated clicks from creating duplicate applications. Decisions lock the membership row. Pending can become verified or rejected; rejected students can apply again. Audit events preserve the decision trail.

Room capacity and unconfirmed floor values are nullable. The user confirmed that 017 is ground floor, 1xx is first floor and 2xx is second floor, so source inventory uses that convention. An administrator records confirmed values through RoomController. This update uses the same scheduling write lock as booking approval, so approval reads consistent room metadata.

Exercise

Sign in as a student, request Co-Dev membership and verify it using the assigned club head. Reopen the student profile and confirm the verified badge. Try to book a room with the same student account: the server must still refuse. Then try to review membership using an unrelated club head and confirm that it is denied.

Completion check

Explain the difference between authentication, account role, membership status and club-specific authority. Show the database uniqueness constraint, the decision transaction and the tests proving that membership verification never grants booking permission.